

Virne Solver Trace in Simulation and RL Training

Environment layering and what each layer “owns”

Yes — Virne effectively operates with **two environment layers** when you use RL solvers (like `pg_mlp`, `ppo_*`, etc.), and they serve different purposes.

The **simulation/system environment** is created once by the system (`BaseSystem.from_config`) as a `SolutionStepEnvironment(p_net, v_net_simulator, ...)`. It controls the event stream (VN arrivals/leaves), holds the authoritative physical network state, and expects the solver to return a **complete Solution** per VN request. You can see `BaseSystem.from_config` constructing `SolutionStepEnvironment(...)` and the solver instance, then wrapping them in `OnlineSystem`.

1

That system environment returns each “problem instance” as an observation dict with **deep copies** of the current VN and PN (`{'v_net': copy.deepcopy(self.v_net), 'p_net': copy.deepcopy(self.p_net)}`), so the solver can safely plan without mutating the authoritative PN. 2

The **solver-internal RL environment** is instantiated **per VN embedding attempt** inside the solver wrapper (`InstanceAgent.solve` / `learn_with_instance`). This environment is one of the classes in `instance_rl_environment.py` (e.g., `PlaceStepInstanceREnv`, `JointPRStepInstanceREnv`). It exposes a Gym-style step interface where the agent takes **actions** (e.g., choose a physical node id), builds observations via the feature constructor, and computes step rewards via the reward calculator. The per-instance env is created in `InstanceAgent.solve` via `instance_env = self.InstanceEnv(p_net, v_net, ...)`. 3

So your statement is correct with a clean phrasing:

- **System env:** evolves the simulation across many VN requests, consumes `Solution` objects (`env.step(solution)`), and commits/rollbacks resources. 4
- **Instance RL env:** is created inside the solver for each VN request, consumes actions (`env.step(action)`), and constructs a `Solution` sequentially. 5

Evaluation-time trace in `OnlineSystem.run`

This is the trace during **simulation execution** (what happens when `OnlineSystem.run()` is called). This is primarily **inference/decoding**, not learning.

System-level call order

1. `OnlineSystem.run()` (system loop) calls:
2. `self.ready()`

3. `instance = self.env.reset(seed)`
4. `solution = self.solver.solve(instance)`
5. `next_instance, _, done, info = self.env.step(solution)`
6. repeats until done. ¹
7. `SolutionStepEnvironment.reset(seed)` prepares the event stream and returns `get_observation()`, which is `{v_net: copy, p_net: copy}`. ⁶
8. `SolutionStepEnvironment.step(solution)` consumes the solver's returned solution and:
 9. checks constraint violations / admission control thresholds,
 10. if success: calls `controller.deploy(v_net, p_net, solution)`,
 11. if failure: rolls back PN based on `place_result / route_result / early_rejection`,
 12. records metrics and advances to the next event via `transit_obs()`. ⁷

What `solver.solve(instance)` resolves to for RL solvers

For a concrete RL solver like `PgMlpSolver`, the class declaration is:

```
class PgMlpSolver(InstanceAgent, PGSolver):
    ...
```

and it wires the instance env class:

- `InstanceAgent.__init__(self, PgMlpInstanceREnv)`
- `PGSolver.__init__(...)` ⁸

Because `InstanceAgent` is listed first, Python's method resolution order means `solve()` comes from `InstanceAgent.solve` unless overridden. For `pg_mlp`, it is not overridden, so `OnlineSystem.run()` calls `InstanceAgent.solve`. ⁹

Inside `InstanceAgent.solve`: the per-instance RL rollout is hidden behind a "searcher"

`InstanceAgent.solve(instance)` does:

- extract `v_net, p_net` from the instance dict,
- construct `instance_env = self.InstanceEnv(p_net, v_net, controller, recorder, ...)`,
- return `self.searcher.find_solution(instance_env)`. ³

The key idea: **the solver makes multiple actions internally** while building the solution, but those actions happen inside `searcher.find_solution(...)`, not at the system level.

Searcher's internal rollout loop (why you don't see "actions" at the system layer)

`RLSolver.eval(...)` builds a searcher (greedy/sampling/beam/...) via `get_searcher(...)`. You can see the searcher factory selecting a strategy like `GreedySearcher`, `SingleSampleSearcher`, `BeamSearcher`, etc. ¹⁰

For greedy decode, the per-instance action loop looks like:

- `obs = instance_env.get_observation()`
- repeat:
- compute mask: `instance_env.generate_action_mask()`
- tensorize obs: `preprocess_obs_func(obs, device)`
- choose action from `policy.act(...)` (masked) with `sample=False`
- `obs, reward, done, info = instance_env.step(action)`
- return `instance_env.solution` ¹¹

So evaluation-time control flow is:

```
OnlineSystem.run
-> SolutionStepEnvironment.reset (gets VN+PN copies)
-> InstanceAgent.solve
    -> create PlaceStepInstanceREnv / JointPRStepInstanceREnv
    -> searcher.find_solution
        -> loop: get_observation -> policy.act -> instance_env.step(action)
        -> returns a Solution
-> SolutionStepEnvironment.step(solution) (deploy/rollback, advance events)
```

All the "RL actions" happen in the solver-internal env loop, not in the system env loop.

Training-time trace where rollouts are stored and parameters updated

Your question explicitly asks for: *actions* → *rollout saved* → *update actor/critic*. That occurs in **training**, which Virne runs through `solver.learn(...)`, typically called inside `BaseSystem.ready()` before `OnlineSystem.run()` starts evaluating.

Where training is triggered

`BaseSystem.ready()` checks `isinstance(self.solver, RLSolver)`. If true and `num_train_epochs > 0`, it calls:

- `self.solver.learn(self.env, num_epochs=num_train_epochs)`
- then `self.solver.eval()` to enter decoding mode. ¹²

So learning is not normally done inside `OnlineSystem.run()`; it's done in `ready()` first.

Training call order in the code

At the top level:

1. `RLSolver.learn(env, num_epochs)` (RLSolver)
2. chooses `learn_singly` if not distributed, otherwise `learn_distributedly`. ¹³
3. `InstanceAgent.learn_singly(env, num_epochs)` (InstanceAgent)
4. `instance = env.reset(...)`
5. loop:
 - `solution, instance_buffer, last_value = learn_with_instance(instance)`
 - `merge_instance_experience(...)`
 - if global buffer is large enough: `self.update()`
 - `instance, reward, done, info = env.step(solution)` ¹⁴

This is important: even in training, the **outer loop** advances the *system environment* by doing `env.step(solution)`, while the **inner loop** advances the *instance RL environment* by doing `instance_env.step(action)`.

The inner per-instance rollout loop (where actions happen and buffer is filled)

`InstanceAgent.learn_with_instance(instance)` builds a fresh per-instance RL env and rolls out an episode:

- `instance_env = self.InstanceEnv(p_net, v_net, ...)`
- `instance_obs = instance_env.reset()`
- repeat:
 - `tensor_obs = self.preprocess_obs(instance_obs, device)`
 - `action, action_logprob = self.select_action(tensor_obs, sample=True)`
 - `value = self.estimate_value(tensor_obs)`
 - `next_obs, reward, done, info = instance_env.step(action)`
 - `instance_buffer.add(obs, action, reward, done, logprob, value, next_obs)`
- return `(solution, instance_buffer, last_value)` ¹⁵

Ownership in that loop:

- `learn_with_instance` is **InstanceAgent**
- `preprocess_obs`, `select_action`, `estimate_value` are **RLSolver** (initialized in `RLSolver.__init__`, used by agent wrapper) ¹⁶
- `instance_env.step` is **PlaceStepInstanceREnv / JointPRStepInstanceREnv** ⁵
- `instance_buffer.add` is **RolloutBuffer** ¹⁷

Turning episode transitions into returns/advantages

After the per-instance env episode ends, `merge_instance_experience(...)` computes learning targets and merges the episode into the solver's global buffer:

- `instance_buffer.compute_returns_and_advantages(last_value, gamma, gae_lambda, method=...)`
- `self.buffer.merge(instance_buffer)` ¹⁸

The `return/advantage` computation is implemented in `RolloutBuffer.compute_returns_and_advantages(...)` and supports GAE, MC, TD. ¹⁹

When the update happens and what “gets updated”

In `learn_singly`, the update trigger is:

- `if self.buffer.size() >= self.target_steps: loss = self.update()` ²⁰

For **Policy Gradient**, the update method is:

- `PGSolver.update()` (PGSolver)
- preprocesses stored observations
- calls `evaluate_actions(...)` to get log-probabilities
- computes REINFORCE loss: `loss = -(action_logprobs * returns).mean()`
- calls `update_grad(loss)` which executes `optimizer.step()`
- clears buffer ²¹

For **A2C**, the update uses advantages and a critic loss, so it really updates both actor+critic:

- actor loss: `-(logprob * advantage)`
- critic loss: `mse(return, value)`
- entropy bonus, etc. ¹³

So the “actor and critic if used get updated” depends on algorithm:

- **PGSolver**: practically updates the **actor**; critic is not in the loss. ¹³
- **A2CSolver / PPOSolver**: explicitly updates **actor + critic** (critic appears in the loss). ²²

The actual optimizer step and gradient logic is centralized in `RLSolver.update_grad(loss)`, which does `zero_grad → backward → clip → step` (and syncs gradients if distributed). ¹³

Method-by-method ordered trace with parent class attribution

Below is the ordered trace you asked for, written as a **call chain** with “method owner” explicitly annotated.

Evaluation iteration inside OnlineSystem.run

```
OnlineSystem.run                                [OnlineSystem/BaseSystem]
  -> env.reset(seed)                             [BaseEnvironment.reset /
SolutionStepEnvironment]
  -> get_observation() -> {v_net copy, p_net copy}
  -> solver.solve(instance)                       [InstanceAgent.solve] (for
pg_mlp-like solvers)
  -> instance_env = InstanceEnv(p_net,v_net)      [PlaceStepInstanceREnv /
JointPRStepInstanceREnv __init__]
  -> searcher.find_solution(instance_env)         [Searcher subclass]
  -> obs = instance_env.get_observation()
[InstanceREnv.get_observation]
  -> feature_constructor.construct               [FeatureConstructor]
  -> obs['action_mask']=...                      [InstanceREnv]
  -> loop until done:
    -> tensor_obs = preprocess_obs(...) [RLSolver.preprocess_obs
(bound to TensorConvertor)]
    -> action = policy.act(...)                 [Policy (ActorCritic)]
    -> instance_env.step(action)                [PlaceStepInstanceREnv.step
or JointPRStepInstanceREnv.step]
    -> controller.node_mapper.place             [Controller]
    -> controller.link_mapper...                [Controller] (when needed)
    -> reward_calculator.compute               [RewardCalculator via
InstanceREnv.compute_reward]
  -> return instance_env.solution
  -> env.step(solution)                         [SolutionStepEnvironment.step]
  -> if success: controller.deploy(...)          [Controller]
  -> else: rollback_for_failure(...)             [BaseEnvironment]
  -> count_and_add_record(...)                  [BaseEnvironment/Recorder /
Counter]
  -> transit_obs() -> next event                 [BaseEnvironment]
  -> repeat
```

Every statement above is directly backed by the corresponding implementations of `OnlineSystem.run`, `InstanceAgent.solve`, searcher loops, instance env stepping, and system env stepping. ²³

Training iteration that includes rollout saving and parameter update

```
BaseSystem.ready                                [BaseSystem]
  -> solver.learn(system_env, num_epochs)         [RLSolver.learn]
  -> learn_singly(system_env, num_epochs)
[InstanceAgent.learn_singly]
  -> instance = system_env.reset(...)
[SolutionStepEnvironment.reset]
  -> loop VN instances:
```

```

-> learn_with_instance(instance)
[InstanceAgent.learn_with_instance]
    -> instance_env = InstanceEnv(...) [PlaceStepInstanceREnv/
JointPRStepInstanceREnv]
    -> obs = instance_env.reset()          [InstanceREnv.reset ->
RLBaseEnv.reset]
    -> loop steps:
        -> tensor_obs = preprocess_obs [RLSolver binding]
        -> action, logprob = select_action(...)
[RLSolver.select_action]
        -> value = estimate_value(...)
[RLSolver.estimate_value]
        -> next_obs, r, done = instance_env.step [Instance RL env]
        -> instance_buffer.add(...)
[RolloutBuffer.add]
        -> return solution, instance_buffer, last_value
    -> merge_instance_experience(...)
[InstanceAgent.merge_instance_experience]
    -> instance_buffer.compute_returns_and_advantages(...)
[RolloutBuffer]
        -> self.buffer.merge(instance_buffer)
    -> if self.buffer.size >= target_steps:
        -> self.update()                    [PGSolver.update or
A2CSolver.update, etc.]
        -> evaluate_actions(...)
[RLSolver.evaluate_actions]
        -> loss = ...
        -> update_grad(loss)                [RLSolver.update_grad]
        -> optimizer.step()
    -> system_env.step(solution)
[SolutionStepEnvironment.step]

```

This is the exact “actions → rollout saved → update” sequence you asked for. ²⁴

Why `solver.solve` is not the update boundary

One subtle but critical refinement to your wording:

- In Virne, `solver.solve(instance)` is an **inference/decode operation** (build a solution for that one VN), even though it internally steps an instance RL env multiple times. ²⁵
- The **parameter update** step is driven by the **training loop** (`solver.learn → learn_singly → update`) and is typically invoked from `BaseSystem.ready()` before `OnlineSystem.run()` starts the evaluation simulation loop. ¹²

That’s why you correctly “see” rollouts and update logic in `learn_with_instance`, `merge_instance_experience`, and `update()`, not in `solve()`.

If you want, I can generate the same trace but specific to **JointPRStepInstanceREnv** vs **PlaceStepInstanceREnv** (the per-step logic differs: joint `controller.place_and_route(...)` vs two-stage `node_mapper.place` then `link_mapper.link_mapping`). ⁵

¹ ¹² ²³ ²⁴ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/system/base_system.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/system/base_system.py

² ⁴ ⁶ ⁷ <https://raw.githubusercontent.com/GeminiLight/virne/main/virne/core/environment.py>
<https://raw.githubusercontent.com/GeminiLight/virne/main/virne/core/environment.py>

³ ⁹ ¹⁴ ¹⁵ ¹⁸ ²⁰ ²⁵ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/instance_agent.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/instance_agent.py

⁵ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/instance_rl_environment.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/instance_rl_environment.py

⁸ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/reinforcement_learning/mlp_solver.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/reinforcement_learning/mlp_solver.py

¹⁰ ¹¹ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/searcher.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/searcher.py

¹³ ¹⁶ ²¹ ²² https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/rl_solver.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/rl_solver.py

¹⁷ ¹⁹ https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/buffer.py
https://raw.githubusercontent.com/GeminiLight/virne/main/virne/solver/learning/rl_core/buffer.py